MOAR Console — visual walkthrough

A flip-through of each console view, captured live from the marimo app, with the intent of each view annotated underneath. Generated from console @**911a820**.

```
Startup › { Strategy › [ Pick components, Vault & Matrix ], Configuration }
Flow › [ Land, Health, Migrate ]
Analyze
```

These are real screenshots of the live UI. The data-driven panels (Health, Analyze) show their honest pre-audit / no-data states because the demo stack is not running — the gate never bluffs a pass, so an un-run audit reads as unmeasured, not green. Regenerate this PDF with **bash flipthrough/build.sh** after a console change.

Security Data Works

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Reference architectures — a known-good starting point

Validated patterns from the book (Appendix C). Match one, then tune; the constraint and anti-pattern checks still apply to whatever you land on.

Lean single-engine lakehouse

A small team (2-3 engineers), cost-led, mixed analyst workloads with no heavy concurrency. One engine over shared Iceberg covers most of what a SOC asks before a second engine earns its keep.

Storage seaweedfs · **Catalog** polaris · **Ingest**

Workload-optimized multi-engine

Mixed real-time detection and threat hunting with 3-5 engineers: a fast aggregation engine (ClickHouse) beside the Iceberg-native default (DataFusion), both over the same tables.

Storage seaweedfs · **Catalog** polaris · **Ingest**

Air-gapped on-prem

An on-prem or data-sovereignty mandate that rules out cloud-managed services. Self-hosted object store + REST catalog + a federating MPP engine, all inside the boundary.

Storage dell_ecs · **Catalog** polaris · **Ingest** nifi · **Query** trino · **Schema** ocsf

Cost-aggressive route-by-value

High-volume, low-value-heavy telemetry where the binding constraint is cost: route-by-value at ingest drops 70-90% of the volume before it lands, on flat no-egress storage.

Storage wasabi · **Catalog** polaris · **Ingest** cribl · **Query** datafusion · **Schema** ocsf

Intent. The constraint-first decision flow. You declare the binding constraints first (deployment, team size, vendor posture, workload, compliance, cost), and the funnel narrows the full component catalog to a fit-justified shortlist — disqualify / caution / favor per candidate, with a reachable-N/M count and a best-fit pick. This is the public half of the Capability Matrix's method (constraint-conditional weighting) shown live and free; the scored ranking — which specific tool wins for your workload archetype and by how much — is the paid Matrix at /matrix.

Security Data Works

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Startup | Flow | Analyze

Strategy | Configuration

Pick components | Vault & Matrix

Architecture Strategy & OKF Vault

This panel reads the project1 strategy vault as a **Google Open Knowledge Format (OKF v0.1)** bundle — a directory of markdown files whose type: frontmatter and `[[wikilinks]]` form a knowledge graph (Google Cloud, published 2026-06-12). Decisions (`MDR-xxxx`) and Assumptions (`A-xx`) are read straight off disk by `okf_reader`, so the stack you pick above stays coupled to the recorded *why*. Tolaria indexes this same vault for the agent host; this app reads the files directly rather than through Tolaria's read-only MCP server.

Search vault: Search decisions & assu

Decision Records (MDRs) — 32 match:

- **MDR-0027** — Adopt augment-vs-replace decision-path scoring (`Accepted`, `2026-06-17`) · `MDR-0027-augment-vs-replace-decision-path-scoring.md`
- **MDR-0010** — Pipelines (C4) criterion taxonomy — 9 criteria (`Accepted`, `2026-05-25`) · `MDR-0010-pipelines-criterion-taxonomy.md`
- **MDR-0023** — Engines criterion addition — native_ip_type_support (`Accepted`, `2026-05-25`) · `MDR-0023-engines-native-ip-type.md`
- **MDR-0015** — Default 6-month revalidation cadence, faster on benchmark triggers (`Accepted`, `2026-05-25`) · `MDR-0015-revalidation-cadence.md`
- **MDR-0031** — Ratify the illustrative-worked-example exception to the public/paid line (`Accepted`, `2026-06-17`) · `MDR-0031-illustrative-worked-example-public-exception.md`
- **MDR-0004** — Score per archetype with conditional weights, not absolute weights (`Accepted`, `2026-05-25`) · `MDR-0004-archetype-conditional-weights.md`
- **MDR-0005** — Cap criteria per component at 7–14 dimensions (`Accepted`, `2026-05-25`) · `MDR-0005-dimensionality-3-to-14.md`

Intent. Two consultant-mode surfaces plus the evidence runner. The Strategy Vault (OKF) and the Capability Matrix scorecard are paid/consultant IP — in the public default mode the scorecard shows no scores and points to /matrix; with PAID_MODE on, the consultant sees the per-criterion scored view loaded from the private vault (never the repo). The thesis-evidence verbs re-prove the program's claims live against the stack.

Security Data Works

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Startup | Flow | Analyze

Strategy | Configuration

Preview source: Zeek conn.log -> Network Activity (4001)

Configuration — raw → OCSF · Network Activity (4001)

Pick a source and watch the deployed mapping turn one **raw event** into an **OCSF record**, field by field. The ⚠ rows are the semantic traps a name-based mapper gets wrong — where the schema, not the field name, decides the answer.

Raw event (synthetic; row 1 of 8):


```
{
 "ts": "1767225601.0",
 "uid": "CbenA",
 "id.orig_h": "10.0.1.21",
 "id.orig_p": "49210",
 "id.resp_h": "10.0.0.10",
 "id.resp_p": "443",
 "proto": "tcp",
 "service": "ssl",
 "duration": "2.31",
 "orig_bytes": "520",
 "resp_bytes": "8400",
 "conn_state": "SF",
 "missed_bytes": "0",
 "history": "ShADadFf",
 "orig_pkts": "6",
 "orig_ip_bytes": "840",
 "resp_pkts": "10",
 "resp_ip_bytes": "8760"
}
```


Intent. Configure the stack spec, validate the OCSF transform (VRL) before provisioning, and deploy / tear-down the MOAr stack locally via Pulumi. The deploy is authorized by the data-health gate (shown here as a compact verdict chip; the full breakdown lives in Flow › Health) — you can't deploy onto an incoherent selection without logging an override.

Security Data Works

MOAr Stack Control Plane

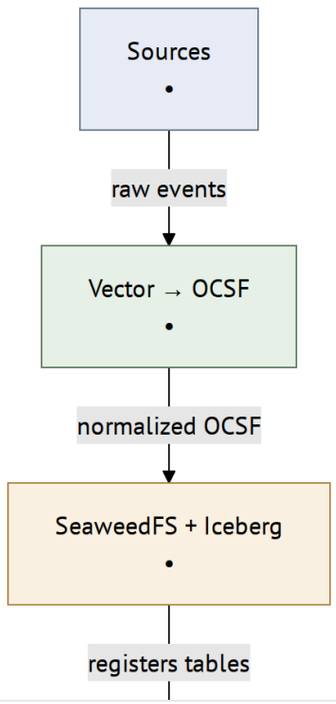
Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Startup | Flow | Analyze

Land | Health | Migrate

Land — pipeline topology

How an event travels through the selected stack — Source(s) → Route (ingest, OCSF-normalize) → Lakehouse (storage + Iceberg) brokered by the Catalog → Engine(s) → Present.



Intent. The pipeline topology — your selected stack drawn as the path an event actually travels: Source → Route (ingest, OCSF-normalize) → Lakehouse (storage + Iceberg) brokered by the Catalog → Engine(s) → Present. A node with no live telemetry shows '—', never a fabricated 'up'; throughput appears only when a real signal carries it.

Security Data Works

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Startup | Flow | Analyze

Land | Health | Migrate

Data-Health Gate

☒ Not certified — a required layer failed: Layer 2 — stack reachable.

- ☒ **Config integrity (compatible selection)** — pass
- ☒ **Spec persisted** — pass
- ○ **Layer 1 — source health** — unmeasured
- ☒ **Layer 2 — stack reachable** — fail
- ○ **Layer 3 — data-quality audit** — unmeasured
- ○ **Layer 4 — cross-tool gap analysis** — unmeasured

Unproven layers are labeled, never shown as a pass; run the Layer-3 audit (Flow → Health) to turn it green. A green gate is the deploy/inspect authorization, not a slide.

Data Health & Schema Validation (Layers 1 & 3)

Run the data-quality audit (Layer 3, on the selected table) and the source-health audit (Layer 1, across the namespace's sources) together:

Parquet CRC checksum integrity (bit-

DuckLake tombstone silent-

Small-file compaction threshold

Intent. The data-health gate — the console's center of gravity. It refuses to certify the foundation GREEN until each measurable layer actually passes: source health (L1), data quality (L3), cross-tool coverage (L4), plus the cluster-5 deepenings — cross-engine answer-equality, OCSF round-trip mapping fidelity, and flow reconciliation. Unproven layers are labeled, never shown as a pass; a proven layer decays to 'stale' if it isn't re-run. A GREEN gate is the deploy authorization, not a slide.

Security Data Works

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Startup | Flow | Analyze

Land | Health | Migrate

Migrate — intent-driven

Pick a migration **intent**; the cockpit expands to focused, verifiable direction — the steps, the swap cost and how to back out, and the live data-health check that proves the move didn't change the answer.

Migration intent: Stand up the lakehouse alongside the SIEM (augment, don't replace)

Migrate — Stand up the lakehouse alongside the SIEM (augment, don't replace)

Run the open lakehouse next to the SIEM and prove parity before you move any read path. Nothing is cut over, so this is the cheapest possible first move — the SIEM stays authoritative while you build confidence that the same questions get the same answers from the lake.

Direction

> **Step 1**
> Tee ingest: fan one copy of telemetry into the SIEM (unchanged) and one into the lakehouse via your selected pipeline, landing as OCSF on SeaweedFS.

> **Step 2**
> Register the landed tables in Polaris and point DataFusion at them.

Intent. The intent-driven migration cockpit. Pick one of six migration intents and the panel expands to focused, verifiable direction: the steps, the real swap-cost and how to back out (read off each component's reversibility), and the live data-health check that proves the move didn't change the answer — cross-engine answer-equality for an engine swap, the swap-store/catalog/router checks for those tiers, flow reconciliation for a route change. The open architecture makes being wrong cheap; this makes that concrete.

ANALYZE

Security Data Works

MOAr Stack Control Plane

Reactive administrator cockpit for the Modular Open Architecture (MOAr) Stack.

Startup | Flow | Analyze

Detections — hunts over the landed OCSF (aggregate-safe)

Worked detections run over the landed OCSF table. Each result is an **aggregate** — a bounded grouping key plus counts — never a raw event row (real telemetry is a prompt-injection surface). *Worked over the Zeek 4001 sample library (a planted beacon + a high-egress source); the live version runs run_detections.py over the landed `ocsf.network_activity` table.*

*

***C2 beacon — low-byte repeated outbound to a rare destination** · `T1071` · 1 finding(s)

`10.0.1.77 → 203.0.113.66` — connections 3 · avg bytes out 84.0

*

***Data exfiltration — high outbound volume by source** · `T1048` · 1 finding(s)

`10.0.1.200` — total bytes out 15000 · flows 3

*

***Credential stuffing / password spray — failed-auth burst from one source** · `T1110` · 1 finding(s)

`203.0.113.99` — failed attempts 6 · distinct users 6

Analyze — log analysis

Intent. Aggregate-only analysis over the loaded OCSF table — row and per-class/activity counts, top-N sources, field population, time span — plus the Iceberg metadata inspector. It never renders a raw telemetry row: real security data is a prompt-injection and control-char surface, so only counts and low-cardinality keys leave the query, and source values are control-char-stripped and length-bounded.